CrisisVision AI

Multimodal AI Crisis Detection & Emergency Intelligence System

Project Overview:

The CrisisAI Multimodal Disaster Intelligence System is an Artificial Intelligence platform designed to detect and analyze disaster-related events from multimodal social media data, specifically images and tweet text. The system uses deep learning to classify disasters such as floods, hurricanes, fires, earthquakes, and other crisis events, estimate their severity level, detect possible locations, and generate an AI-powered emergency response report.

In modern disaster management, social media platforms like Twitter (X) play a crucial role in providing real-time information during emergencies. People often post images and messages during disasters before official agencies report them. However, this information is highly unstructured, noisy, and difficult to analyze manually at scale.

CrisisAI addresses this challenge by combining:

- Computer Vision (for analyzing disaster images)
- Natural Language Processing (for understanding tweet text)
- Multimodal Deep Learning (for combining both signals)
- Named Entity Recognition (NER) (for detecting locations)
- Generative AI (Groq LLM) (for generating emergency response reports)

The system converts unstructured social media posts into structured crisis intelligence, enabling emergency responders to quickly understand:

- What disaster is happening
- Where it is happening
- How severe it may be
- What actions should be taken

This allows faster disaster response and improved situational awareness." module, enabling users to upload study materials and query them using context-aware AI.

SCENARIO 1 : Real-Time Disaster Detection from Social Media

During natural disasters such as floods, hurricanes, or wildfires, affected individuals often post images and messages on social media describing the situation around them.

For example, a tweet might contain:

> "Water levels rising quickly in Mumbai streets after heavy rain"
>
> along with an image showing flooded roads.

Manually monitoring thousands of such posts is extremely difficult for emergency teams.

The CrisisAI system automatically processes these posts by:

Reading the tweet text.

- Analyzing the attached disaster image.
- Combining both signals using a multimodal deep learning model.
- Predicting the type of disaster (Flood, Fire, Hurricane, etc.).
- Estimating the confidence level of the prediction.
- Extracting potential location information from the text.

The result is a structured prediction such as:

> Disaster Type: Flood
>
> Confidence: 92%
>
> Detected Location: Mumbai

This enables emergency systems to detect crises earlier using real-time social media signals.

SCENARIO 2: Automated Disaster Severity Analysis

Not all disaster-related posts indicate severe emergencies. Some may describe minor incidents, while others may represent major crises requiring immediate response.

CrisisAI analyzes multimodal data to estimate disaster severity levels such as:

- Low Severity (minor flooding or small fire)
- Moderate Severity (localized damage)
- High Severity (major disaster affecting large areas)

By analyzing both the image features and text context, the model can determine whether a situation likely requires immediate emergency response.

For example:

Tweet Content	Prediction
"Small fire contained near highway"	Low severity
"Massive wildfire spreading quickly"	High severity

This classification helps prioritize emergency response resources effectively.

SCENARIO 3: AI-Generated Emergency Response Report

After detecting the disaster type and location, the system generates an AI-powered emergency report using a Groq LLM (Llama 3.3).

The AI produces a structured response including:

- Situation Overview
- Immediate Response Actions
- Affected Areas

Example generated report:

Situation Overview:

A flooding event has been detected in Mumbai with high confidence based on social media reports and image analysis.

Immediate Actions Required:

- Deploy rescue teams to flooded streets.
- Monitor water levels and issue public safety warnings.
- Coordinate with local authorities for evacuation planning.

Affected Areas:

Mumbai coastal regions and nearby urban areas appear impacted.

This transforms raw social media signals into actionable emergency intelligence.

Key Objectives of CrisisAI

The primary objectives of the CrisisAI project include:

- Detect disasters automatically from social media data.
- Combine image and text information using multimodal deep learning.
- Identify disaster type and severity level.
- Extract location information from tweet text.
- Generate AI-driven emergency response reports.
- Provide real-time disaster intelligence for emergency responders.

Unique Features of the Project

The CrisisAI system demonstrates several advanced AI capabilities:

- Multimodal AI model combining BERT + ResNet50
- Disaster classification using deep learning
- Location extraction using Named Entity Recognition
- AI-generated emergency summaries using Groq LLM
- Real-time crisis detection workflow
- Structured disaster intelligence pipeline

High-Level System Workflow

The system operates through the following pipeline:

1. User inputs tweet text and disaster image
2. Text is processed using BERT tokenizer
3. Image is processed using ResNet50 CNN
4. Features from both modalities are combined
5. Deep neural network predicts disaster type
6. NER model extracts location entities
7. Groq LLM generates an emergency response report
8. Final structured output is displayed to the user

ARCHITECTURE OVERVIEW

The CrisisAI Multimodal Disaster Intelligence System is designed using a modular architecture that separates user interaction, data input handling, multimodal inference, and AI-assisted report generation into clearly defined layers. This architecture allows the system to process disaster-related social media content efficiently while maintaining simplicity and flexibility for user-driven analysis.

At the frontend layer, the system uses Streamlit, a Python-based framework for building interactive machine learning applications. The Streamlit interface provides a user-friendly environment where users can manually provide disaster-related inputs in two ways. Users can either enter a single tweet along with an associated image, or they can upload a CSV file containing multiple tweets for batch analysis. This design ensures that the system remains flexible and usable for both quick testing scenarios and larger datasets.

When the user submits input through the Streamlit interface, the system forwards the data to the backend processing modules responsible for multimodal inference. The backend is implemented entirely in Python, where pretrained deep learning models are loaded during application startup. These models have already been trained on disaster-related datasets, and the system uses pretrained model weights downloaded from external sources. Therefore, the application performs model inference only, without any training occurring during runtime.

The core of the system is the Multimodal Disaster Classification Engine, which processes both textual and visual information from the input data. The tweet text is first processed using a BERT-based Natural Language Processing (NLP) model, which converts the text into contextual embeddings representing semantic meaning and disaster-related patterns. Simultaneously, the input image is processed using a ResNet50 Convolutional Neural Network (CNN) that extracts high-level visual features related to disaster scenes such as floods, fires, hurricanes, or damaged infrastructure.

The textual features extracted from the BERT model and the visual features extracted from the ResNet50 model are combined using a multimodal feature fusion mechanism. This combined representation enables the model to capture relationships between textual descriptions and visual evidence, improving classification accuracy compared to single-modality approaches.

Once the multimodal features are fused, they are passed into a fully connected neural network classification layer that predicts the type of disaster event. The system is capable of identifying several disaster categories such as Flood, Hurricane, Fire, Earthquake, and Other Crisis Events. Along with the predicted disaster label, the model outputs a confidence score, which represents the probability of the prediction.

Following disaster classification, the system applies a Location Extraction Module based on Named Entity Recognition (NER) techniques. This module analyzes the tweet text to identify geographic entities such as city names, regions, or landmarks mentioned in the tweet. Extracted location information helps provide contextual awareness regarding where the disaster might be occurring.

The final stage of the architecture is the AI-assisted Emergency Report Generation Module, which uses a Groq Large Language Model (LLM) to generate a structured disaster response report. The system constructs a prompt containing the predicted disaster type, detected location entities, and prediction confidence. This information is sent to the Groq LLM, which produces a detailed textual response summarizing the situation and suggesting potential response actions.

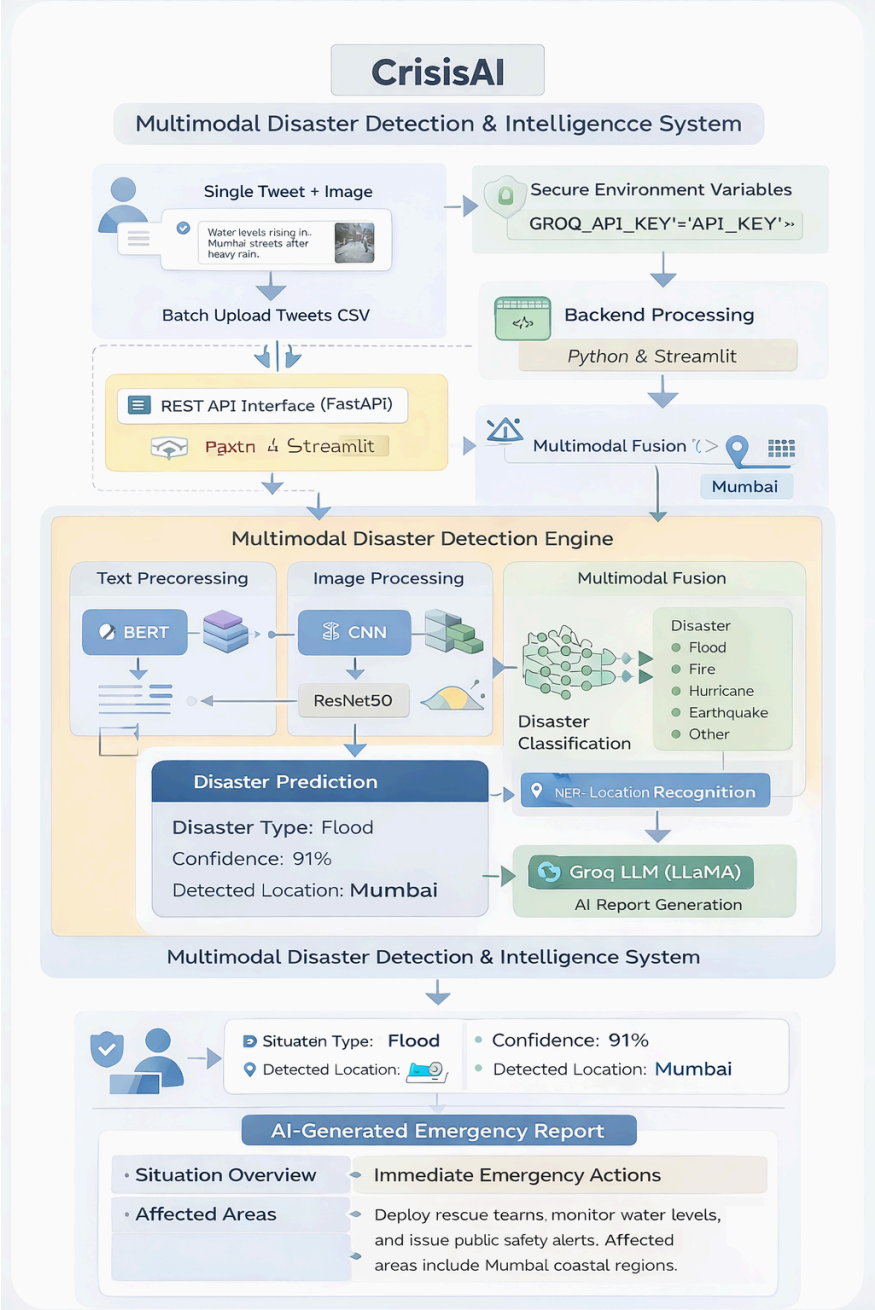
The generated report typically includes:

- A situation overview describing the detected disaster event
- Immediate response actions that emergency teams may consider
- Potentially affected locations derived from the tweet content

To ensure secure integration with external AI services, the system stores sensitive credentials such as Groq API keys in environment variables or configuration files, preventing exposure of API credentials in the source code.

Because the system uses pretrained models with downloaded weights, the architecture focuses primarily on efficient inference and structured output generation rather than model training. This design simplifies deployment while maintaining strong predictive performance.

The modular architecture also allows future expansion of the system, such as incorporating real-time social media data ingestion, disaster severity estimation, geospatial visualization of detected locations, automated alert systems, and integration with emergency response platforms. Overall, the CrisisAI architecture provides a flexible and scalable framework for analyzing disaster-related social media content using multimodal artificial intelligence.


CrisisAI
Multimodal Disaster Detection & Intelligencce System
Single Tweet + Image
Water levels rising in. Mumhai streets after heavy rain.
Batch Upload Tweets CSV
Secure Environment Variables
GROQ_API_KEY'='API_KEY'»
Backend Processing
Python & Streamlit
REST API Interface (FastAPI)
Paxtn 4 Streamlit
Multimodal Fusion
Mumbai
Multimodal Disaster Detection Engine
Text Precoressing
Image Processing
Multimodal Fusion
BERT
CNN
ResNet50
Disaster
Flood
Fire
Hurricane
Earthquake
Other
Disaster Classification
Disaster Prediction
Disaster Type: Flood
Confidence: 91%
Detected Location: Mumbai
NER- Location Recognition
Groq LLM (LLaMA)
AI Report Generation
Multimodal Disaster Detection & Intelligence System
Situaten Type: Flood
Detected Location:
Confidence: 91%
Detected Location: Mumbai
AI-Generated Emergency Report
Situation Overview
Immediate Emergency Actions
Affected Areas
Deploy rescue tearns, monitor water levels, and issue public safety alerts. Affected areas include Mumbal coastal regions.

CORE TECHNOLOGIES

- **Streamlit (Frontend Interface)**: The CrisisAI system uses Streamlit to build an interactive frontend interface for the disaster detection application. Streamlit allows users to easily interact with the AI model through a simple web-based dashboard. Users can either paste a single tweet and upload an associated image for analysis or upload a CSV file containing multiple tweets for batch processing. The interface dynamically displays model predictions, detected locations, confidence scores, and AI-generated emergency reports.

- **Python (Core Development Language):** Python serves as the primary programming language used throughout the project. It enables integration of machine learning models, natural language processing pipelines, image processing modules, and generative AI APIs. Python's extensive ecosystem makes it suitable for building end-to-end AI applications.

- **BERT-base-uncased (Text Understanding Model):** The system uses BERT-base-uncased to analyze tweet text and extract contextual semantic features. BERT converts raw tweet sentences into high-dimensional embeddings that represent the meaning of the text. These embeddings help the model understand disaster-related language such as references to floods, fires, hurricanes, and other emergency events.

- **ResNet-50 (Image Feature Extraction Model):** For visual analysis, the system uses ResNet-50, a deep convolutional neural network pretrained on large-scale image datasets. ResNet-50 extracts high-level visual features from uploaded images, identifying patterns associated with disaster scenes such as flooding, fire damage, or destroyed infrastructure.

- **Multimodal Fusion Model:** The CrisisAI model combines text embeddings from BERT and visual features from ResNet-50 through a multimodal fusion architecture. This fusion layer integrates information from both modalities to produce a unified representation, enabling the system to classify disaster events more accurately than using text or images alone.

- **XLM-RoBERTa (Named Entity Recognition Model):** To extract location information from tweet text, the system uses XLM-RoBERTa, a multilingual transformer model designed for Named Entity Recognition (NER). This model identifies geographic entities such as cities, countries, and regions mentioned in tweets, helping determine where a disaster event might be occurring.

- **Llama 3.3 70B (Groq Generative AI):** The system integrates Llama 3.3 70B running on Groq infrastructure for generating AI-powered emergency reports. After disaster classification and location extraction, the system constructs a structured prompt containing the predicted disaster type and detected locations. The Groq LLM then generates a detailed response that includes a situation overview, emergency response recommendations, and potentially affected areas.

- **Pandas (Batch Data Processing):** The Pandas library is used for processing CSV datasets containing multiple tweets. It allows the system to efficiently load, iterate, and analyze tweet records during batch inference.

- **dotenv (Environment Configuration):** The dotenv library is used to securely manage environment variables such as the Groq API key. This prevents sensitive credentials from being hardcoded in the source code.

MODEL STACK

The CrisisAI system uses a multimodal AI pipeline composed of specialized models for text understanding, image analysis, location extraction, and generative AI reporting.

Component	Model
Text Processing	BERT-base-uncased
Image Processing	ResNet-50
Named Entity Recognition	XLM-RoBERTa
Generative AI Report Engine	Llama 3.3 70B (Groq)

COMPONENT-WISE ARCHITECTURE

Component	Description
User Interface (Streamlit)	Provides an interactive web-based interface built using Streamlit where users can analyze disaster-related social media posts. Users can either paste a single tweet with an associated image or upload a CSV file containing multiple tweets for batch analysis. The interface displays predicted disaster type, model confidence, detected locations, and AI-generated emergency reports.
Input Handling Module	Responsible for receiving and validating user inputs from the Streamlit interface. This module supports two input modes: single tweet with image upload and batch tweet analysis through CSV files. It ensures the provided data is correctly formatted before sending it to the inference pipeline.

Text Processing Module (BERT-base-uncased)	Processes tweet text using the pretrained BERT-base-uncased transformer model. The module converts raw tweet text into contextual embeddings that capture semantic meaning and disaster-related information present in the tweet. These embeddings are later used in the multimodal classification model.
Image Processing Module (ResNet-50)	Extracts visual features from uploaded images using a pretrained ResNet-50 convolutional neural network. This module identifies visual patterns related to disaster scenes such as floods, fires, damaged infrastructure, or storms. The extracted visual feature vectors are passed to the multimodal fusion layer.
Multimodal Fusion Layer	Combines textual embeddings generated by BERT and visual features extracted by ResNet-50 into a unified feature representation. This multimodal representation allows the system to leverage both textual context and visual evidence when predicting the disaster type.
Disaster Classification Module	Uses the fused multimodal features to predict the type of disaster event. The classification model outputs a disaster category such as Flood, Fire, Hurricane, Earthquake, or Other Events along with a confidence score representing the probability of the prediction.
Named Entity Recognition Module (XLM-RoBERTa)	Extracts location-related entities from tweet text using the pretrained XLM-RoBERTa model. This module identifies geographic names such as cities, countries, or regions mentioned in the tweet to provide contextual information about where the disaster might be occurring.
AI Report Generation Module (Groq LLM – Llama 3.3 70B)	Generates a structured emergency response report using the Groq LLM. The system constructs a prompt containing the predicted disaster type, extracted location entities, and confidence score. The LLM produces a human-readable response describing the situation overview, possible risks, and recommended emergency actions.
Batch Processing Module (Pandas)	Handles CSV datasets containing multiple tweets for batch analysis. This module loads the dataset using Pandas, iterates through tweet records, and processes each tweet through the disaster detection pipeline.

Pretrained Model Loader	Loads pretrained model weights for BERT, ResNet-50, and XLM-RoBERTa during application initialization. This ensures the system performs inference using previously trained models without retraining during runtime.
Environment & Configuration Manager (dotenv)	Manages environment variables and securely loads API credentials such as the Groq API key. This prevents sensitive information from being stored directly in the source code.
Error Handling & Validation Layer	Handles invalid inputs, model loading issues, API failures, or runtime errors. This module ensures the system returns structured messages instead of crashing when unexpected issues occur.

PRE-REQUISITES

The successful development, execution, and deployment of the CrisisAI Multimodal Disaster Detection and Emergency Intelligence System require several hardware, software, and technical prerequisites. These prerequisites ensure stable execution of the multimodal deep learning models, tweet processing pipeline, image analysis system, location extraction module, and AI-powered emergency report generation system.

The following requirements must be satisfied before running the project.

1. Python Environment Setup

Python is the primary programming language used for implementing the CrisisAI system. It supports integration of deep learning frameworks, natural language processing models, image processing libraries, and generative AI APIs.

Required Version

- Python 3.10 or above
- **Recommended:** Python 3.11

Purpose

- Multimodal model inference (BERT + ResNet50)
- Tweet text preprocessing and tokenization
- Image feature extraction
- Named Entity Recognition using XLM-RoBERTa
- Integration with Groq LLM for AI report generation
- Streamlit application development

2. Virtual Environment (venv)

A virtual environment is recommended to isolate project dependencies and avoid conflicts with system-wide Python packages.

Benefits

- Prevents dependency conflicts
- Ensures reproducible environments
- Simplifies project deployment and sharing

Create Virtual Environment

```
python -m venv .venv
```

Activate Environment

Windows: .venv\Scripts\activate

Linux / macOS: source .venv/bin/activate

3. Required Python Libraries

All dependencies required for the CrisisAI system must be installed before running the application. The libraries support deep learning inference, multimodal data processing, frontend development, and AI integration.

Key Libraries Used

Library	Purpose
streamlit	Frontend interface for interacting with the disaster detection system
torch	Deep learning framework used for loading pretrained models and performing inference
torchvision	Provides pretrained computer vision models such as ResNet-50
transformers	HuggingFace library used for loading BERT and XLM-RoBERTa models
pandas	Data processing and CSV batch tweet handling

numpy	Numerical computations and tensor operations
pillow	Image loading and preprocessing
scikit-learn	Machine learning utilities and evaluation tools
groq	Integration with Groq API for Llama 3.3 AI report generation
python-dotenv	Secure management of environment variables
uvicorn	ASGI server used for running backend services
fastapi	Optional backend API framework for model interaction

Additional dependencies include utilities for HTTP requests, tokenization, and dataset handling as listed in the project requirements file.

4. Installation of Dependencies

All required packages can be installed using the provided requirements.txt file.

Installation Command: pip install -r requirements.txt

These libraries enable the execution of the multimodal AI inference pipeline and Streamlit interface.

5. Groq API Setup

The system uses Groq's Llama 3.3 70B model to generate AI-powered emergency response reports based on disaster predictions.

Steps:

- Create an account on the Groq platform
- Generate an API key
- Create a .env file in the project root directory
- Add the following line:
  GROQ_API_KEY=your_api_key_here

Purpose:

- Generate disaster situation summaries
- Provide emergency response recommendations
- Produce structured crisis intelligence reports

6. Streamlit Frontend Setup

The CrisisAI system uses Streamlit for its interactive user interface.

Purpose:

- Accept tweet text input
- Accept disaster image uploads
- Support CSV upload for batch tweet analysis
- Display disaster prediction results
- Display detected locations and AI reports

Run the Application: streamlit run app.py

7. Pretrained Model Availability

The CrisisAI system relies on pretrained models with downloaded weights for inference.

Required Models

- BERT-base-uncased (text understanding)
- ResNet-50 (image feature extraction)
- XLM-RoBERTa (location entity recognition)

These models are automatically downloaded from HuggingFace Transformers when the system runs for the first time.

8. Input Data Requirements

The system does not automatically fetch data from social media platforms. Instead, users provide the input data manually.

Supported Input Modes:

- Single Post Analysis
    - Paste a tweet text
    - Upload an associated disaster image

- Batch Tweet Analysis
    - Upload a CSV file containing multiple tweets

The system processes each input through the multimodal disaster detection pipeline.

PROJECT FLOW

1. Environment Setup and Dependency Configuration

Activity 1.1: Install Python 3.10+ and create a dedicated virtual environment (venv) to ensure clean and isolated dependency management.

Activity 1.2: Install required libraries including PyTorch, Transformers, Streamlit, Pandas, Groq API SDK, Pillow, NumPy, and python-dotenv using the requirements.txt file.

Activity 1.3: Configure environment variables by creating a .env file and securely storing the GROQ_API_KEY required for AI-powered emergency report generation.

Activity 1.4: Verify successful installation by running a test script to ensure that pretrained models such as BERT-base-uncased, ResNet-50, and XLM-RoBERTa load correctly.

Activity 1.6: Configure Git and GitHub for version control, collaborative development, and academic project submission.

2. Disaster Tweet Dataset Preparation

Activity 2.1: Prepare a dataset containing disaster-related tweets and corresponding images collected from publicly available datasets or research repositories.

The dataset may include disaster scenarios such as:

- Floods
- Hurricanes
- Wildfires
- Earthquakes
- Storm damage

Activity 2.2: Ensure that tweet text and associated images accurately represent disaster events.

Activity 2.3: Validate dataset quality by checking text clarity, image relevance, and label consistency.

3. Pretrained Multimodal Model Setup

Activity 3.1: Load the pretrained BERT-base-uncased model for text processing and contextual embedding generation.

Activity 3.2: Load the pretrained ResNet-50 convolutional neural network for extracting visual features from disaster images.

Activity 3.3: Implement preprocessing pipelines for both modalities:

Text preprocessing:

- Tokenization
- Attention mask generation
- Input encoding for BERT

Image preprocessing:

- Image resizing
- Normalization
- Tensor conversion

Activity 3.4: Extract feature embeddings from both models.

Activity 3.5: Implement a multimodal fusion layer that combines textual embeddings and visual features into a unified representation.

4. Disaster Classification Engine Development

Activity 4.1: Design a classification layer that receives fused multimodal features and predicts the type of disaster event.

The model predicts categories such as:

- Flood
- Hurricane
- Fire
- Earthquake
- Other Disaster Events

Activity 4.2: Generate prediction confidence scores indicating the reliability of the classification.

Activity 4.3: Implement inference logic so that pretrained weights are loaded and used for prediction without retraining the model.

Activity 4.4: Ensure error-safe handling for cases where images are missing or text input is invalid.

5. Location Extraction using Named Entity Recognition

Activity 5.1: Integrate the XLM-RoBERTa transformer model for Named Entity Recognition (NER).

Activity 5.2: Process tweet text to detect location-related entities such as:

- City names
- Countries
- Regions
- Landmarks

Activity 5.3: Extract detected locations and store them as structured output for disaster reporting.

Activity 5.4: Validate entity extraction results and filter irrelevant entities.

6. AI Integration using Groq LLM

Activity 6.1: Integrate the Groq API to access the Llama 3.3 70B large language model.

Activity 6.2: Design structured prompt templates containing:

- Predicted disaster type
- Model confidence score
- Extracted locations
- Tweet description

Activity 6.3: Send the structured prompt to the Groq LLM to generate an AI-powered emergency response report.

Activity 6.4: Implement safe response handling and fallback logic in case of API errors.

Activity 6.5: Format the AI response into structured sections such as:

- Situation overview
- Immediate emergency response actions
- Potentially affected areas

7. Streamlit Application Development

Activity 7.1: Develop an interactive user interface using Streamlit.

The interface includes:

- Tweet input text box
- Image upload component
- CSV upload component for batch analysis

Activity 7.2: Implement two modes of operation:

- Single Analysis Mode: User pastes tweet text and uploads a disaster image.
- Batch Analysis Mode: User uploads a CSV file containing multiple tweets.

Activity 7.3: Connect the Streamlit interface with the backend inference pipeline.

Activity 7.4: Display results dynamically including:

- Predicted disaster type
- Model confidence score
- Extracted location entities
- AI-generated emergency report

Activity 7.5: Implement user-friendly features such as loading indicators and error messages.

8. Testing, Debugging, and Optimization

Activity 8.1: Perform unit testing for major modules including:

- Text preprocessing
- Image preprocessing
- Multimodal fusion
- Disaster classification
- NER extraction

Activity 8.2: Validate Streamlit interface functionality and user input handling.

Activity 8.3: Debug common issues such as:

- Model loading errors
- Image preprocessing failures
- Tokenization mismatches
- API connection errors

Activity 8.4: Optimize inference performance by minimizing unnecessary model reloads and caching models.

Activity 8.5: Conduct end-to-end testing using real disaster tweet examples.

MILESTONE 1 – Multimodal Disaster Detection Model Training

This milestone focuses on building and training the core multimodal disaster detection model using a training notebook. The objective of this stage was to develop a deep learning architecture capable of analyzing both tweet text and disaster images simultaneously in order to classify disaster events accurately.

Unlike traditional machine learning models that rely on a single input modality, this system leverages multimodal learning, where textual and visual information are combined to improve prediction performance. Social media posts often contain both text descriptions and images during disaster situations, and integrating both signals helps the model better understand the context of crisis events.

The training process was implemented using the notebook on kaggle:

crisis-detection-training_notebook.ipynb

By the end of this milestone, the system was capable of:

- Processing tweet text using BERT
- Processing disaster images using ResNet-50
- Combining multimodal features
- Training a disaster classification model
- Saving trained model weights for later inference

Activity 1.1: Dataset Preparation

The first step in training the model involved preparing a dataset containing disaster-related tweets and images.

The dataset consisted of:

- Tweet text describing disaster events
- Associated disaster images
- Disaster category labels

Example disaster categories included:

- Flood
- Fire
- Hurricane
- Earthquake
- Other Disaster Events

Each dataset entry contained both textual and visual data, enabling multimodal learning.

Example dataset entry:

Tweet Text: "Massive flooding in the streets after heavy rainfall."

Image: Flooded street image

Label: Flood

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18126 entries, 0 to 18125
Data columns (total 18 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   tweet_id           18126 non-null  int64
 1   image_id           18126 non-null  object
 2   text_info          18126 non-null  object
 3   text_info_conf     18126 non-null  float64
 4   image_info         18126 non-null  object
 5   image_info_conf    18126 non-null  float64
 6   text_human         13807 non-null  object
 7   text_human_conf    13807 non-null  float64
 8   image_human        13807 non-null  object
 9   image_human_conf   13807 non-null  float64
 10  image_damage       3635 non-null   object
 11  image_damage_conf  3635 non-null   float64
 12  tweet_text         18126 non-null  object
 13  image_url          18126 non-null  object
 14  image_path         18126 non-null  object
 15  disaster_type      18126 non-null  object
 16  label              18126 non-null  object
 17  image_full_path    18126 non-null  object
dtypes: float64(5), int64(1), object(12)
memory usage: 2.5+ MB
```

```
final_data=data[['tweet_text','image_full_path','label']]
```

Activity 1.2: Text Feature Extraction using BERT

Tweet text was processed using the BERT-base-uncased transformer model, which is widely used for natural language understanding tasks.

BERT converts raw tweet text into contextual embeddings that capture semantic relationships between words.

Preprocessing steps included:

- Tokenization
- Padding and truncation
- Attention mask generation

```
# Text
encoding = tokenizer(
    row["tweet_text"],
    truncation=True,
    padding="max_length",
    max_length=MAX_LEN,
    return_tensors="pt"
)
```

```
input_ids = encoding["input_ids"].squeeze(0)
attention_mask = encoding["attention_mask"].squeeze(0)
```

These embeddings represent the semantic meaning of the tweet text and help the model understand disaster-related language.

Activity 1.3: Image Feature Extraction using ResNet-50

Disaster images were processed using ResNet-50, a deep convolutional neural network pretrained on large-scale image datasets.

ResNet-50 extracts high-level visual features that represent patterns within the image.

Image preprocessing steps included:

- Image resizing
- Normalization
- Conversion to tensor format

```
image_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])
```

```
# Image
image = Image.open(row["image_full_path"]).convert("RGB")
image = image_transform(image)
```

These extracted visual features represent disaster-related patterns such as flooding, fire damage, or destroyed infrastructure.

Activity 1.4: Multimodal Feature Fusion

After extracting textual and visual features, the next step was to combine them using a multimodal fusion architecture.

The fusion layer integrates:

- Text embeddings from BERT
- Image features from ResNet-50

The combined feature vector provides a richer representation of the disaster event compared to using text or images alone.

This multimodal representation is then passed to a classification layer for prediction.

```python
class MultiModalModel(nn.Module):
    def __init__(self, num_classes):
        super(MultiModalModel, self).__init__()

        # BERT
        self.bert = AutoModel.from_pretrained("bert-base-uncased")

        # ResNet
        self.cnn = models.resnet50(pretrained=True)
        self.cnn.fc = nn.Identity()  # remove classifier

        # Fusion layers
        self.fc1 = nn.Linear(768 + 2048, 512)
        self.dropout = nn.Dropout(0.3)
        self.fc2 = nn.Linear(512, num_classes)

    def forward(self, input_ids, attention_mask, image):
        # Text
        bert_outputs = self.bert(
            input_ids=input_ids,
            attention_mask=attention_mask
        )
        text_features = bert_outputs.last_hidden_state[:, 0, :]  # CLS

        # Image
        image_features = self.cnn(image)

        # Fusion
        combined = torch.cat((text_features, image_features), dim=1)

        x = self.fc1(combined)
        x = torch.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)

        return x
```

Activity 1.5: Disaster Classification Model

A neural network classifier was implemented to predict disaster categories based on the fused multimodal features.

The classifier predicts categories such as:

- Flood
- Fire
- Hurricane
- Earthquake
- Other Disaster Events

The model outputs:

- Predicted disaster type
- Prediction confidence score

Activity 1.6: Model Training and Optimization

The multimodal model was trained using supervised learning techniques.

Training steps included:

- Forward propagation
- Loss computation
- Backpropagation
- Parameter updates

Loss functions and optimization techniques were used to improve prediction accuracy during training.

Model performance was evaluated using validation datasets to ensure the model generalized well to unseen data.

```python
EPOCHS = 3

for epoch in range(EPOCHS):
    model.train()
    total_loss = 0
    correct = 0
    total = 0

    progress_bar = tqdm(train_loader, desc=f"Epoch {epoch+1}")

    for batch in progress_bar:

        optimizer.zero_grad()

        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        images = batch["image"].to(device)
        labels = batch["label"].to(device)

        outputs = model(input_ids, attention_mask, images)

        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

        preds = torch.argmax(outputs, dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

        # Update tqdm bar
        progress_bar.set_postfix({
            "Loss": f"{loss.item():.4f}",
            "Acc": f"{correct/total:.4f}"
        })

    print(f"\nEpoch {epoch+1} Summary:")
    print(f"Avg Loss: {total_loss/len(train_loader):.4f}")
```

```
Epoch 1:  55%|▌▌▌▌▌      | 997/1813 [05:01<03:53,  3.49it/s, Loss=0.0029, Acc=0.9674]/usr/local/lib/python3.12/dist-packages/PIL/Image.py:1047: UserWarn
  warnings.warn(
Epoch 1: 100%|▌▌▌▌▌▌▌▌▌▌| 1813/1813 [09:11<00:00,  3.29it/s, Loss=0.0013, Acc=0.9770]

Epoch 1 Summary:
Avg Loss: 0.0853
Train Accuracy: 0.9770
Epoch 2: 100%|▌▌▌▌▌▌▌▌▌▌| 1813/1813 [07:14<00:00,  4.17it/s, Loss=0.0008, Acc=0.9932]

Epoch 2 Summary:
Avg Loss: 0.0229
Train Accuracy: 0.9932
Epoch 3: 100%|▌▌▌▌▌▌▌▌▌▌| 1813/1813 [07:11<00:00,  4.20it/s, Loss=0.0002, Acc=0.9952]

Epoch 3 Summary:
Avg Loss: 0.0159
Train Accuracy: 0.9952
```

Activity 1.7: Model Weight Saving

After training was completed, the model weights were saved for later inference in the application.

```python
torch.save(model.state_dict(), "multimodal_model_final.pt")
```

```python
import pickle

with open("label_encoder.pkl", "wb") as f:
    pickle.dump(label_encoder, f)
```

Saving the trained model allowed the Streamlit application to load the pretrained weights and perform predictions without retraining.

MILESTONE 2 – Multimodal Inference Pipeline Implementation

After successfully training the multimodal disaster detection model, the next milestone focused on implementing the model inference pipeline that enables the system to analyze new tweet and image inputs. The objective of this stage was to integrate the trained multimodal model into an inference workflow capable of processing user-provided inputs and generating disaster predictions.

Unlike the training stage, which focused on optimizing model parameters using datasets, this phase focuses on efficient inference using pretrained model weights. The trained model is loaded into memory and used to predict disaster events based on user input without performing any additional training.

The multimodal inference pipeline processes two types of inputs:

- Tweet text describing a potential disaster event
- Image associated with the tweet

Both inputs are processed through separate neural networks before being combined to generate a final prediction.

Activity 2.1: Loading Trained Model Weights

The first step in implementing the inference pipeline was loading the trained model weights saved during the training phase.

The saved weights allow the application to perform predictions without retraining the model each time the system runs.

```
model = MultiModalModel(NUM_CLASSES)
model.load_state_dict(torch.load("multimodal_model_final.pt", map_location=device))
model.to(device)
model.eval()
```

Setting the model to evaluation mode ensures that training-specific operations such as dropout are disabled during inference.

Successful loading confirmed that:

- Model architecture matched the trained configuration
- Weight parameters were correctly restored
- The system was ready for inference

Activity 2.2: Tweet Text Preprocessing for Inference

User-provided tweet text must first be converted into a format suitable for the BERT model.

Text preprocessing includes:

- Tokenization
- Padding or truncation
- Attention mask generation

```
def predict(text, image_path):

    # ---- Text Encoding ----
    encoding = tokenizer(
        text,
        truncation=True,
        padding="max_length",
        max_length=128,
        return_tensors="pt"
    )
```

The tokenized inputs are then passed through the BERT model to obtain contextual embeddings representing the tweet's meaning.

These embeddings capture contextual information that helps identify disaster-related language patterns.

Activity 2.3: Image Preprocessing for Inference

Uploaded images are processed using a preprocessing pipeline before being passed to the ResNet-50 model.

Image preprocessing steps include:

- Image resizing to a fixed resolution
- Conversion to tensor format

```
image_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor()
])
```

The processed image tensor is then fed into the ResNet-50 network to extract visual features.

These visual features capture structural patterns associated with disaster scenes.

Activity 2.4: Multimodal Feature Fusion

Once textual and visual features are extracted, they are combined to form a unified multimodal representation.

The fusion layer integrates:

- Text embeddings from BERT
- Image features from ResNet-50

This combined feature vector enables the model to interpret both the semantic meaning of the tweet and the visual evidence from the image.

Feature fusion improves prediction accuracy because the model can cross-reference textual descriptions with visual content.

```python
class MultiModalModel(nn.Module):
    def __init__(self, num_classes):
        super(MultiModalModel, self).__init__()

        self.bert = AutoModel.from_pretrained("bert-base-uncased")

        self.cnn = models.resnet50(pretrained=True)
        self.cnn.fc = nn.Identity()

        self.fc1 = nn.Linear(768 + 2048, 512)
        self.dropout = nn.Dropout(0.3)
        self.fc2 = nn.Linear(512, num_classes)

    def forward(self, input_ids, attention_mask, image):
        bert_outputs = self.bert(
            input_ids=input_ids,
            attention_mask=attention_mask
        )

        text_features = bert_outputs.last_hidden_state[:, 0, :]
        image_features = self.cnn(image)

        combined = torch.cat((text_features, image_features), dim=1)

        x = torch.relu(self.fc1(combined))
        x = self.dropout(x)
        x = self.fc2(x)

        return x
```

Activity 2.5: Disaster Prediction Generation

The fused multimodal representation is passed to the classification layer to generate the final disaster prediction.

The model outputs:

- Predicted disaster type
- Prediction confidence score

Example prediction output:

```
return {
    "disaster_type": disaster_type,
    "confidence": confidence.item(),
    "locations_detected": locations
}
```

The prediction result represents the model's interpretation of the disaster event based on both text and image inputs.

MILESTONE 3 – Location Extraction and AI Emergency Intelligence Integration

After implementing the multimodal disaster detection pipeline, the next milestone focused on enhancing the system by incorporating location extraction and AI-powered emergency intelligence generation. While the multimodal model was capable of identifying the type of disaster from tweet text and images, it did not provide contextual information about where the disaster might be occurring or what response actions might be necessary.

To address this limitation, two major components were integrated into the system:

- Location Extraction using Named Entity Recognition (NER)
- AI-powered Emergency Report Generation using Groq LLM (Llama 3.3 70B)

Activity 3.1: Named Entity Recognition Integration

To identify location information from tweets, a Named Entity Recognition (NER) module was integrated into the system using the XLM-RoBERTa transformer model.

NER models are designed to identify and classify entities in text such as:

- Locations
- Organizations
- Person names
- Dates

For the CrisisAI system, the focus was specifically on detecting location entities that could indicate where a disaster might be occurring.

Example tweet: "Heavy flooding reported near downtown Miami after intense rainfall."

Detected entity:

Location: Miami

This extracted information helps provide geographic context to the disaster event.

Activity 3.2: Tweet Text Processing for NER

The tweet text is passed to the XLM-RoBERTa model for entity recognition.

The model analyzes the text and identifies tokens that correspond to named entities.

```
ner_pipeline = pipeline(
    "ner",
    model="Davlan/xlm-roberta-base-ner-hrl",
    aggregation_strategy="simple"
)

# ============================
# TEXT PREPROCESSING FOR NER
# ============================
def preprocess_for_ner(text):
    text = re.sub(r"http\S+", "", text)
    text = re.sub(r"@\w+", "", text)
    text = re.sub(r"#", "", text)
    text = text.title()
    return text.strip()

def extract_location(text):
    clean_text = preprocess_for_ner(text)
    entities = ner_pipeline(clean_text)

    locations = [
        ent["word"]
        for ent in entities
        if ent["entity_group"] == "LOC"
    ]

    if not locations:
        return "Unknown"

    return list(set(locations))
```

The output typically contains entity labels such as:

LOC – Location

ORG – Organization

PER – Person

For the disaster detection system, the module filters results to retain only location-related entities.

This information is then stored and passed to the next stage of the pipeline.

Activity 3.3: Disaster Context Construction

After extracting the predicted disaster type and location entities, the system constructs a context summary that will be used to generate an emergency response report.

The context contains:

- Predicted disaster type
- Model confidence score
- Extracted locations
- Original tweet description

Example structured context:

Disaster Type: Flood

Confidence Score: 0.91

Detected Location: Miami

Tweet: "Heavy flooding reported near downtown Miami after intense rainfall."

This structured context is essential for generating meaningful AI reports.

Activity 3.4: Prompt Engineering for Emergency Report Generation

To guide the AI model in generating structured disaster reports, carefully designed prompts were created.

The prompt template contains instructions for the LLM to produce a well-structured emergency report.

Prompt engineering ensures that the AI model generates consistent, relevant, and structured responses.

Activity 3.5: Groq LLM Integration

The system integrates the Groq API, which provides access to the Llama-3.3-70B large language model.

This model is responsible for generating the final AI-powered emergency intelligence report.

```python
from groq import Groq
from dotenv import load_dotenv
load_dotenv()

client = Groq(api_key=os.getenv("GROQ_API_KEY"))


def generate_emergency_summary(disaster, confidence, locations, severity="Unknown"):
    prompt = f"""
A multimodal AI crisis detection system analysed a social media post:

Disaster Type: {disaster}
Confidence: {confidence:.4f} ({confidence*100:.1f}%)
Severity Level: {severity}
Locations Detected: {locations}

Generate a professional emergency response report:
1. SITUATION OVERVIEW (1-2 sentences)
2. IMMEDIATE ACTIONS REQUIRED (2-3 bullet points)
3. AFFECTED AREAS (brief note on locations and impact)

Keep under 150 words. Use clear, actionable emergency language.
"""
    chat_completion = client.chat.completions.create(
        messages=[
            {"role": "system", "content": "You are an expert emergency response coordinator with experience in disaster management."},
            {"role": "user", "content": prompt}
        ],
        model="llama-3.3-70b-versatile",
        temperature=0.5,
        max_tokens=250
    )
    return chat_completion.choices[0].message.content.strip()
```

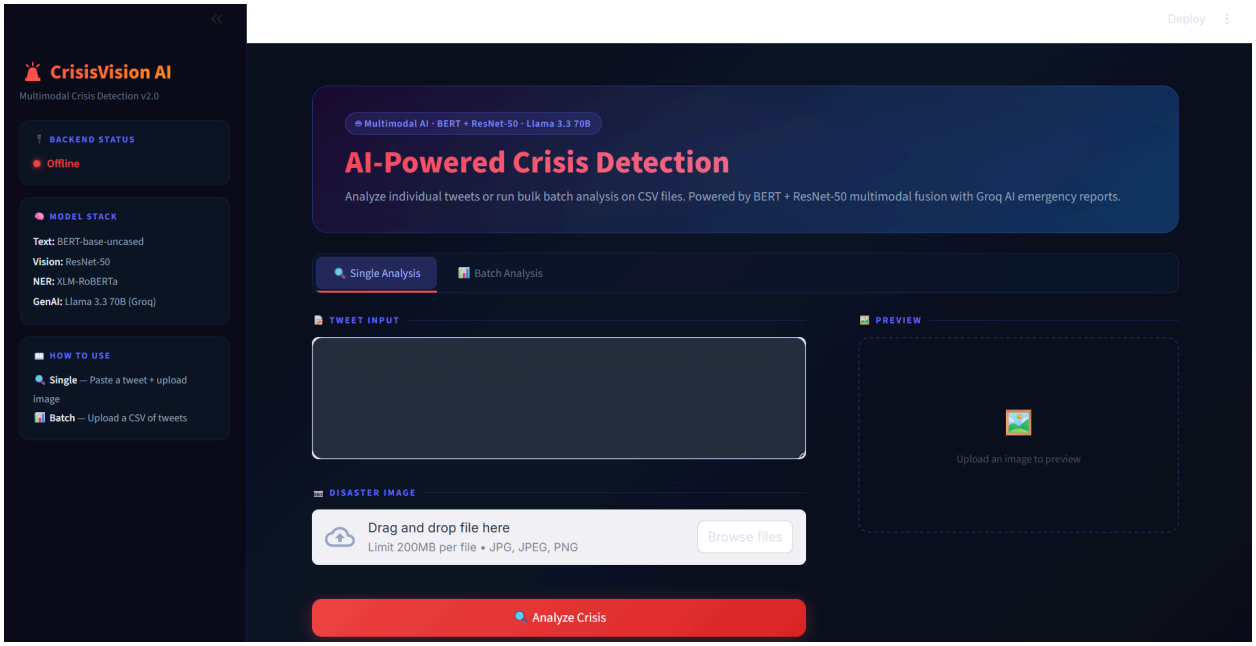
The response generated by the model contains a detailed explanation of the disaster event and suggested emergency response actions.

MILESTONE 4 – Streamlit Disaster Intelligence Application Integration

After developing the multimodal disaster detection model, implementing the inference pipeline, and integrating location extraction and AI-based emergency reporting, the next milestone focused on integrating all system components into a fully functional user-facing application.

The objective of this milestone was to develop an interactive interface that allows users to easily interact with the CrisisAI system without requiring technical knowledge of machine learning models or backend code. To achieve this, the system was implemented as a Streamlit application, which provides a lightweight framework for building interactive machine learning dashboards.

The Streamlit interface acts as the primary interaction layer between the user and the AI system. It allows users to input disaster-related data and receive structured predictions and AI-generated insights in real time.



Activity 4.1: Streamlit Application Initialization

The application interface was built using Streamlit, which allows developers to rapidly create interactive web applications using Python.

The application is launched using the command:

```
streamlit run app.py
```

Once executed, the interface automatically opens in the web browser.

Activity 4.2: User Input Interface Design

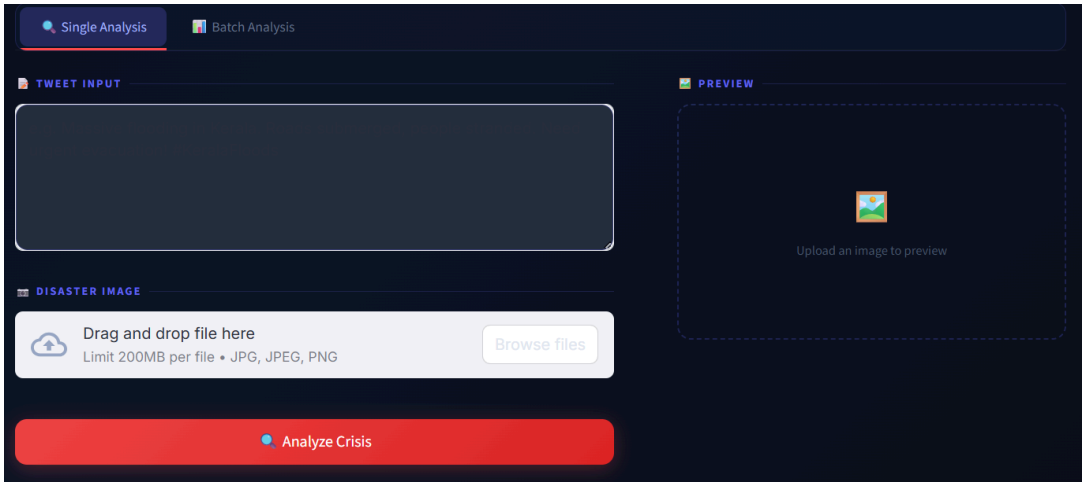
The Streamlit interface was designed to provide a simple and intuitive input system for users.

The application supports two modes of operation:

1. **Single Tweet Analysis**

   Users can analyze an individual social media post by:

   - Entering a tweet text describing a disaster event
   - Uploading a related disaster image

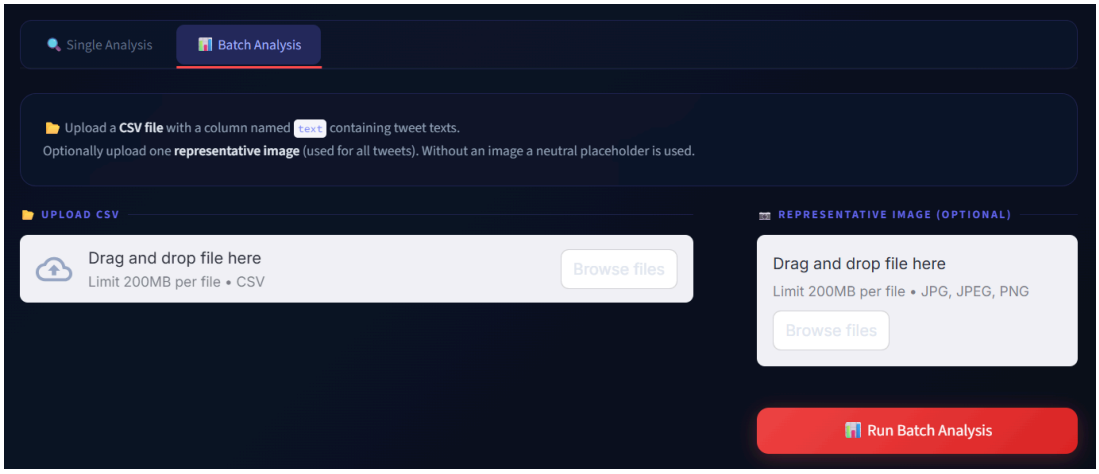


This mode is useful for analyzing individual disaster reports.

2. Batch Tweet Analysis

In addition to single analysis, the application supports batch processing using CSV files.

Users can upload a dataset containing multiple tweets.



The system processes each tweet individually and generates predictions for the entire dataset.

This feature allows users to analyze large collections of social media posts efficiently.

Activity 4.3: Integration with Multimodal Inference Pipeline

The Streamlit interface is directly connected to the multimodal inference pipeline developed in previous milestones.

The workflow for processing a user request is as follows:

- User enters tweet text and uploads image
- Text is processed using the BERT model
- Image is processed using ResNet-50
- Features are combined through the multimodal fusion layer
- Disaster prediction is generated

This integration enables the system to perform real-time disaster classification.

Activity 4.4: Location Extraction Integration

After disaster prediction, the tweet text is passed to the XLM-RoBERTa Named Entity Recognition model to identify location-related entities.

Detected locations are extracted and displayed within the interface.

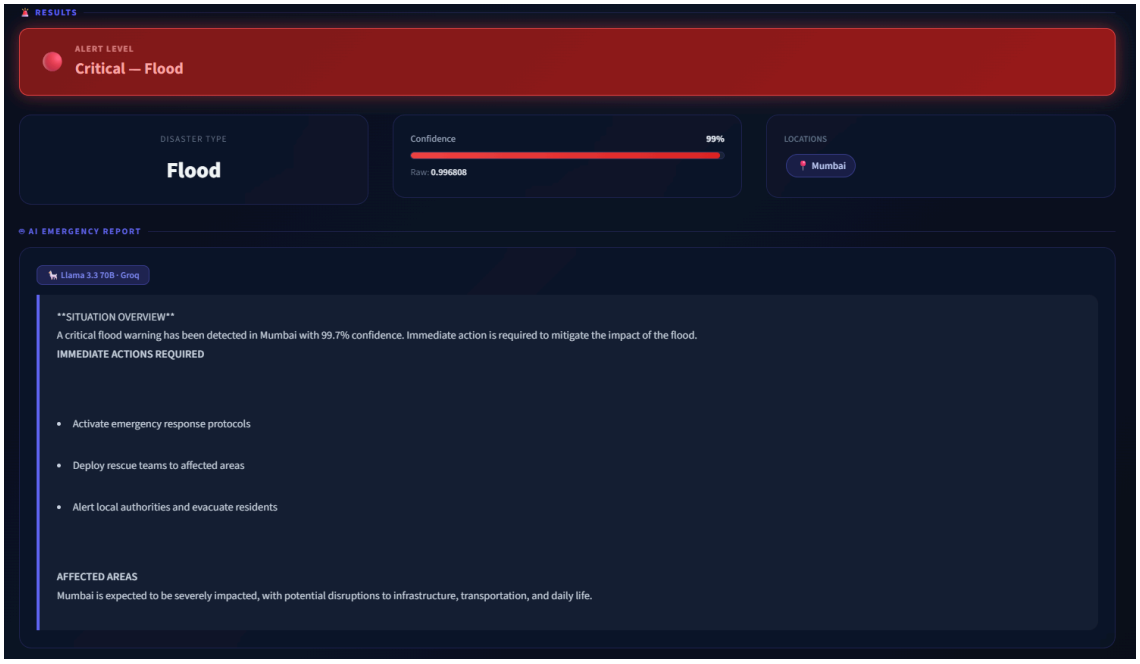
This step helps provide geographic context for disaster events.

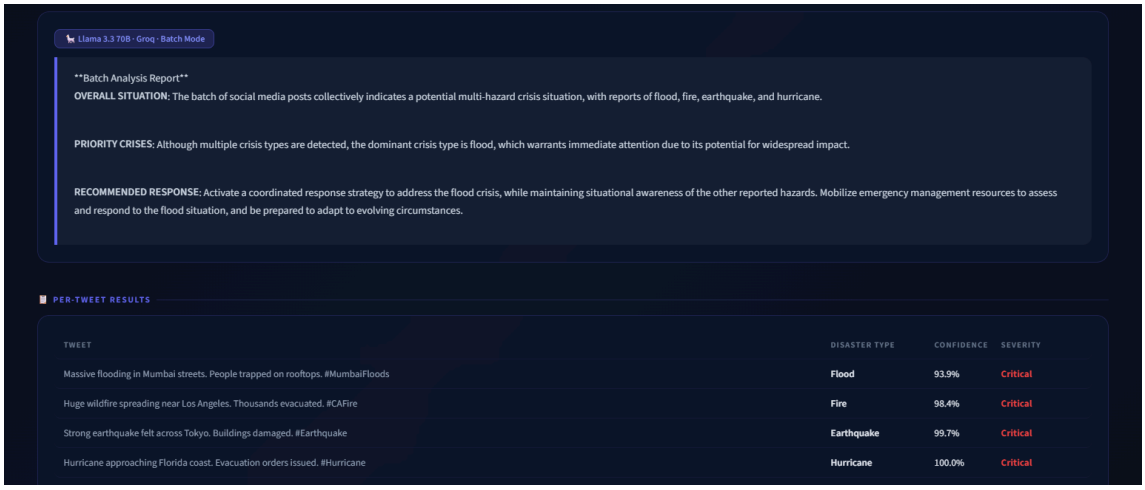
Activity 4.5: AI Emergency Report Display

The extracted information is then sent to the Groq LLM (Llama-3.3 70B) to generate a structured emergency response report.

The Streamlit interface displays the generated AI report in a formatted section.

Activity 4.6: Result Visualization and Output Display





CONCLUSION

The CrisisAI Multimodal Disaster Detection and Emergency Intelligence System demonstrates how artificial intelligence can be used to analyze social media data for disaster awareness and crisis response. The system combines natural language processing, computer vision, and generative AI to interpret disaster-related tweets and images and convert them into meaningful insights.

CrisisAI uses BERT-base-uncased to understand tweet text and ResNet-50 to analyze disaster images. By combining these two modalities through a multimodal model, the system can accurately classify disaster events such as floods, fires, hurricanes, and other emergencies. This approach improves prediction reliability by utilizing both textual descriptions and visual evidence from social media posts.

The system also integrates Named Entity Recognition using XLM-RoBERTa to extract location information from tweets, providing geographic context for detected disaster events. Additionally, Groq's Llama 3.3 70B model is used to generate structured emergency response reports that summarize the situation and suggest potential response actions.

The application is deployed through an interactive Streamlit interface, allowing users to analyze either a single tweet with an image or upload a CSV file for batch tweet analysis. The system then outputs the predicted disaster type, confidence score, detected locations, and an AI-generated emergency report.

Overall, the CrisisAI project demonstrates how multimodal deep learning and generative AI can be integrated to transform unstructured social media content into useful disaster intelligence. With further enhancements such as real-time data integration and geospatial visualization, the system could evolve into a powerful tool for supporting disaster monitoring and emergency response.